

Fundamentals of Software Engineering

Nathaniel Schutta
Dan Vega



... IN THE AGE OF AI



Nathaniel Schutta

🦋 @nts.bsky.social

<https://www.linkedin.com/in/nate-schutta>

<https://ntschutta.io/>



Dan Vega

X @therealdanvega

<https://www.linkedin.com/in/danvega>

<https://danvega.dev>



Part 1: Core Skills

1. Programmer to Engineer
2. Reading Code
3. Writing Code

Part 2: Technical Practices

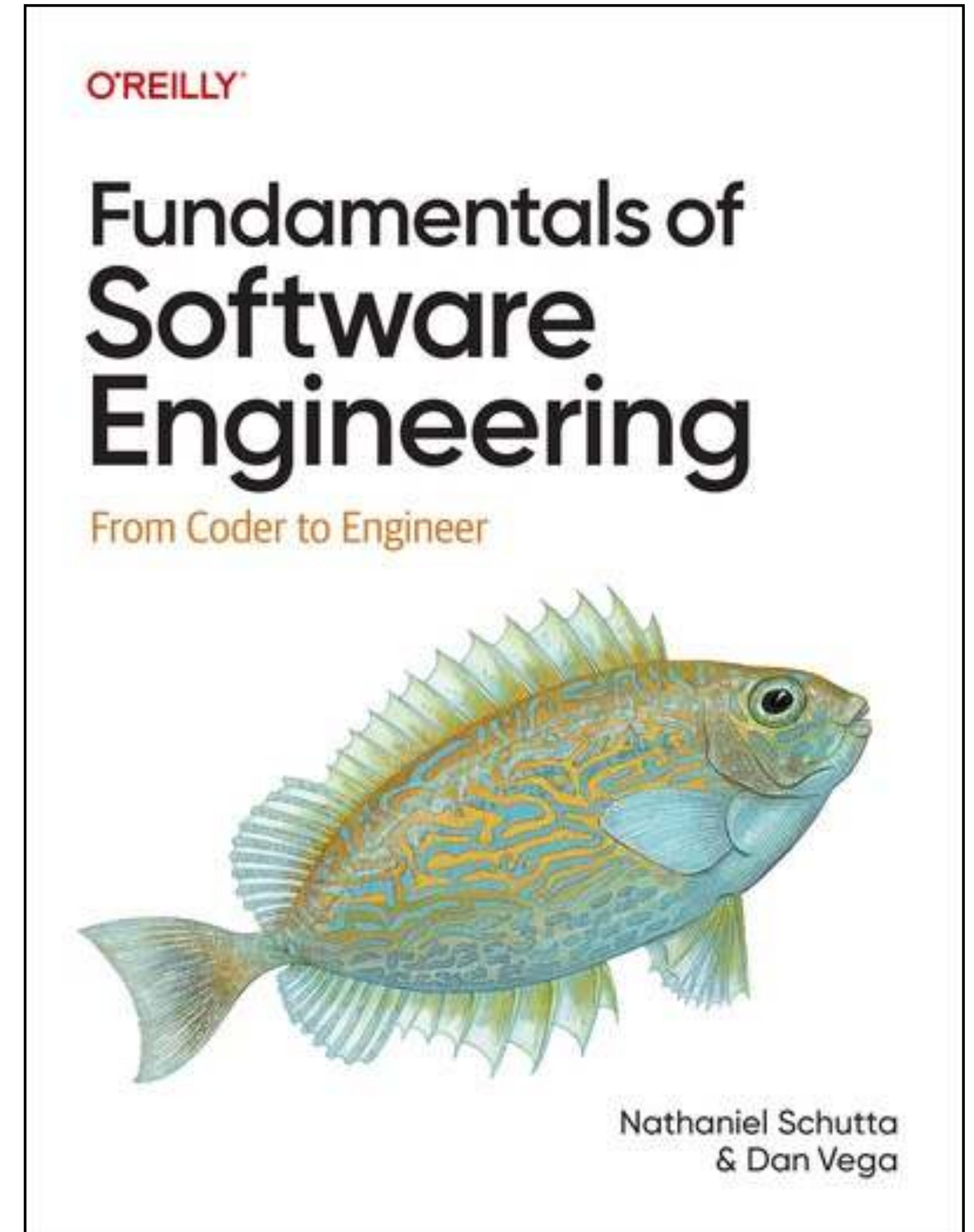
4. Software Modeling
5. Automated Testing
6. Working with Existing Code

Part 3: Application Development and Design

7. User Interface Design
8. Working with Data
9. Software Architecture
10. To Production

Part 4: Professional Development and Growth

11. Powering Up Your Productivity
12. Learning to Learn
13. Mastering Soft Skills in the Tech World
14. Career Management
15. The AI-Powered Software Engineer



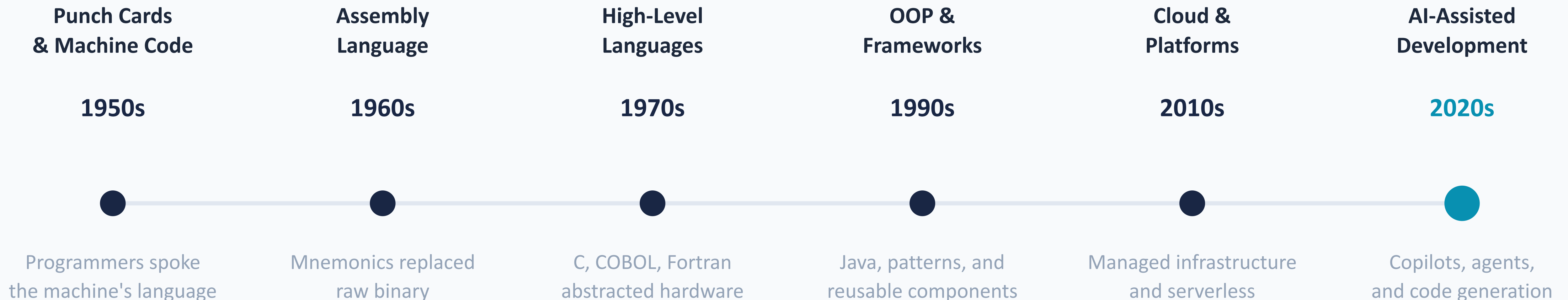
Do the fundamentals still matter?

Where do the
fundamentals matter?

...In the age of AI

A brief history of abstractions

We've always been building layers to make things easier.



Every new abstraction made us more productive — but never eliminated the need to understand what's happening underneath.

Here we go again...

Every few years, something new is going to make developers obsolete.

2000s

IDEs & code generators

"Drag-and-drop will replace coding"

2010s

Frameworks & low-code

"Anyone can build an app now"

2018+

No-code platforms

"Developers are the new buggy whip makers"

2023+

AI code assistants

"Why learn to code at all?"

Yet here we are.

Are our roles changing?

Absolutely!

But first...

What does a Software Engineer
actually do?

~~Write Code. Go Home~~

...right?

What a Software Engineer really does



Requirements
& Planning



System Design
& Architecture



Writing
Code



Testing &
Debugging



Code
Review



Deployment
& CI/CD



Monitoring &
Performance



Communication
& Collaboration



Security &
Compliance



Documentation

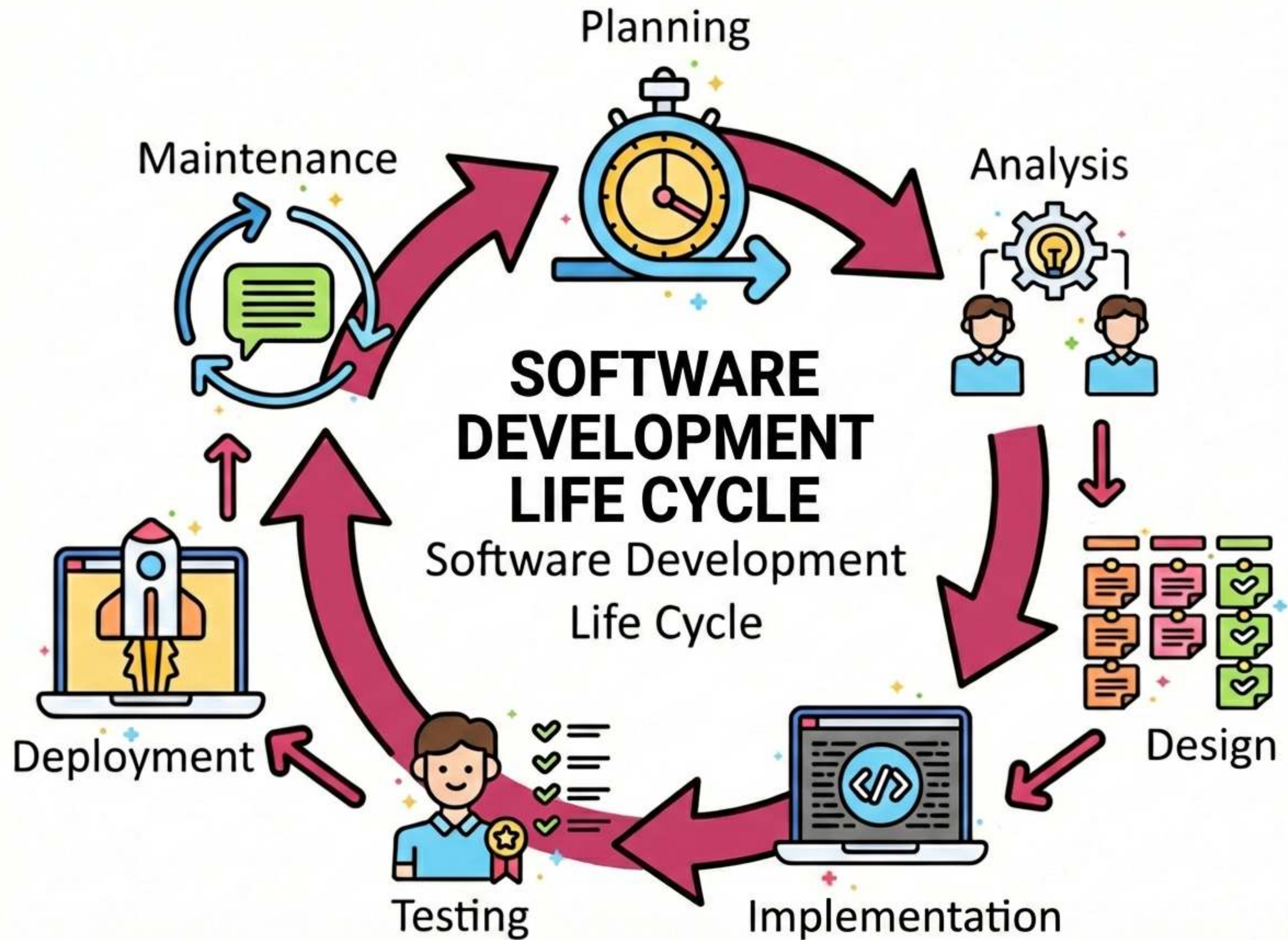


Mentoring &
Leadership



Problem
Solving

Writing code is just one piece of the puzzle



Before we say AI replaces
software engineers...

Let's make sure we understand
what they **actually** do.

Meanwhile...

The Market Panicked.

The software industry has
ebbed and flowed before.

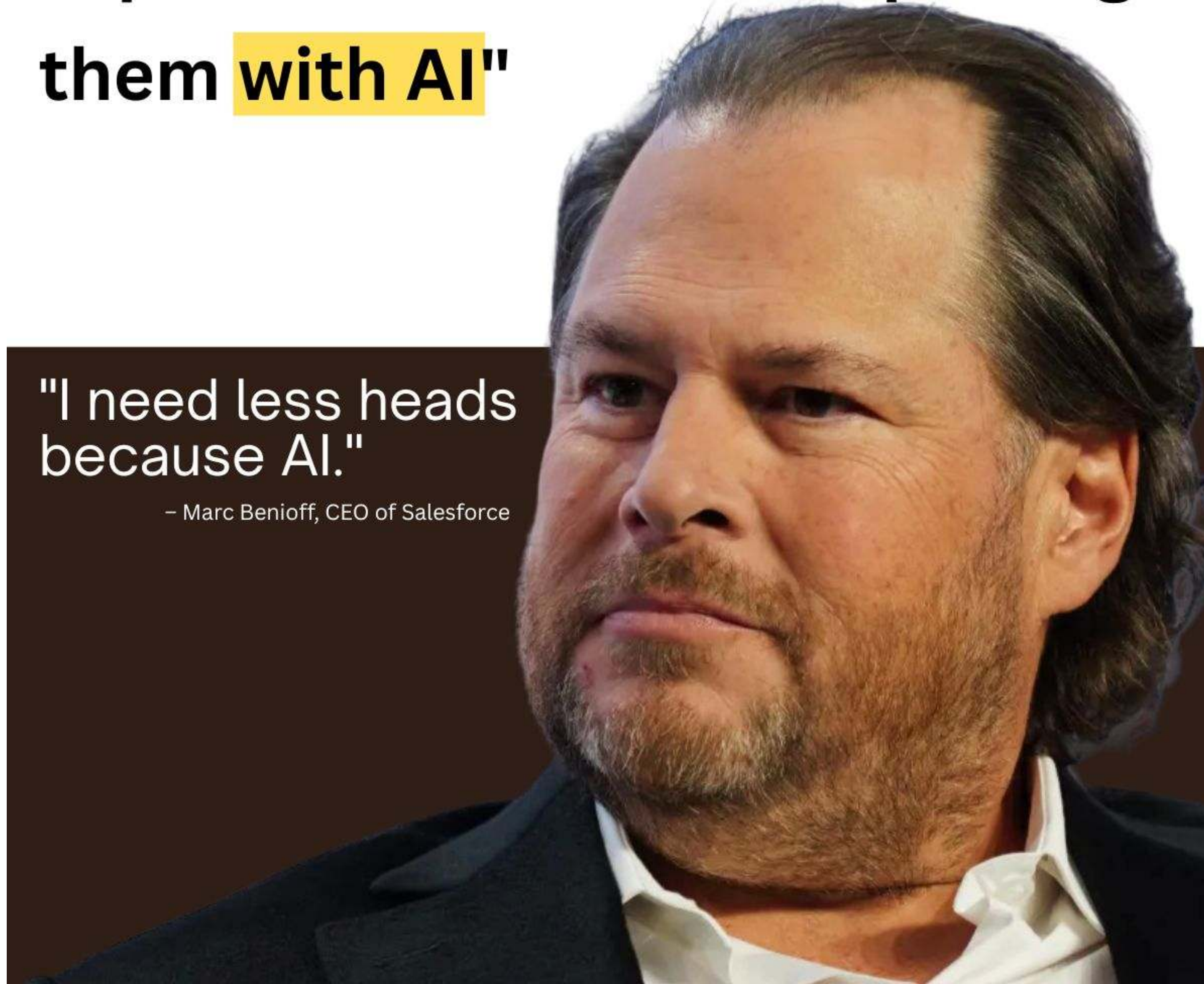
There are always ups and
downs to the job market.

Things seem...
uncertain right now.

**"Salesforce regrets firing 4000
experienced staff and replacing
them with AI"**

**"I need less heads
because AI."**

– Marc Benioff, CEO of Salesforce



" Six months ago, Dario Amodei, the CEO of massive AI company Anthropic, claimed that in half a year, AI would be "writing 90 percent of code." And that was the worst-case scenario; in just three months, he predicted, we could hit a place where "essentially all" code is written by AI."

Staff Software Engineer, Claude Developer Platform (Full Stack)	San Francisco, CA	Apply →
Software Engineer, iOS	San Francisco, CA, New York City, NY, Seattle, WA	Apply →
Software Engineer, Enterprise Foundations	San Francisco, CA New York City, NY	Apply →
Software Engineer, UI Platform	San Francisco, CA New York City, NY	Apply →
Engineering Manager, Claude Developer Platform	San Francisco, CA	Apply →
Software Engineer, Claude Code	New York City, NY; San Francisco, CA; Seattle, WA	Apply →
Engineering Manager, Multicloud and Auth & Identity	New York City, NY; San Francisco, CA; Seattle, WA	Apply →
Software Engineer, Android	San Francisco, CA New York City, NY Seattle, WA	Apply →
Software Engineer, Public Sector	San Francisco, CA New York City, NY Washington, DC	Apply →
Software Engineer, Beneficial Deployments	San Francisco, CA New York City, NY	Apply →
Software Engineer, Platform	New York City, NY; San Francisco, CA; Seattle, WA	Apply →
Software Engineer, Desktop	San Francisco, CA Seattle, WA	Apply →



Gary Marcus
@garymarcus.bsky.social

Have truly lost patience with the New York Times' editorial coverage of AI, which has basically degenerated into industry cheerleading.

They are completely misrepresenting what most academics believe about AGI.

I detail some blunders here:

open.substack.com/pub/garymarc...



Hype, Anthropic's Dario Amodei, the podcasters who love him — and how the New York Times' commentary on AI has degenerated into industry cheerleading

Real journalists do due diligence

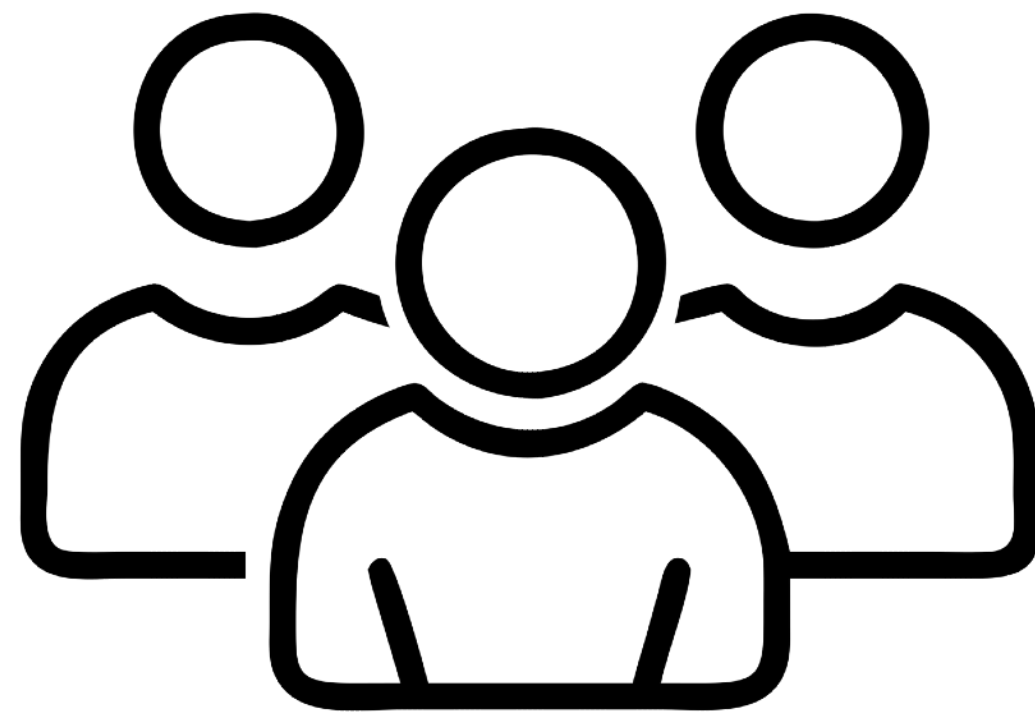
open.substack.com

March 14, 2025 at 10:55 AM Everybody can reply

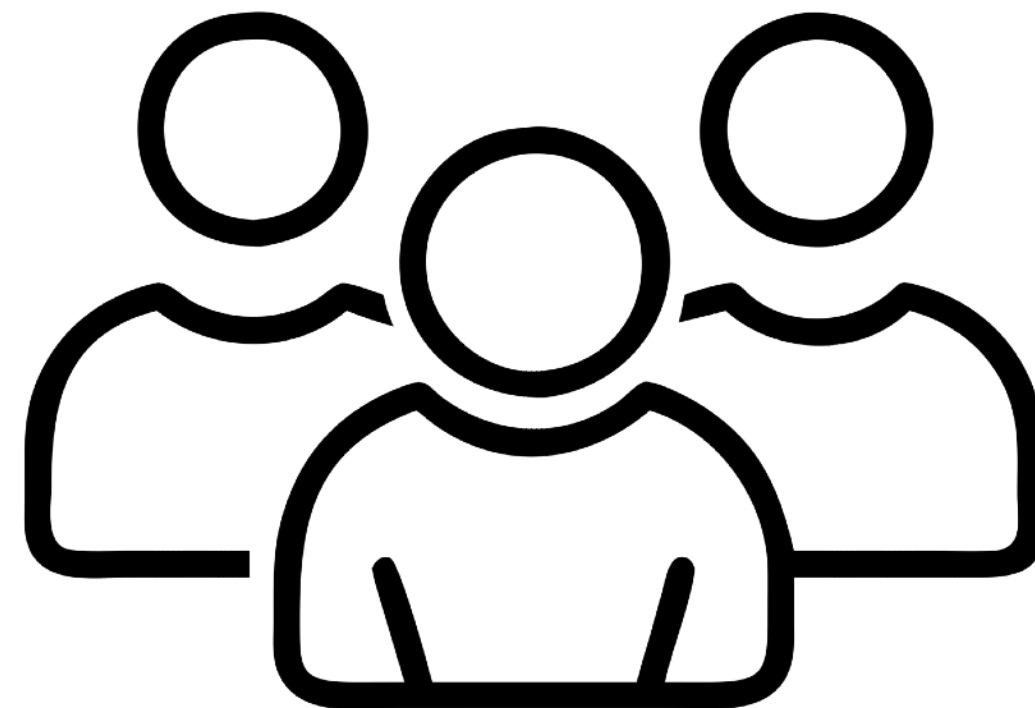
46 reposts 8 quotes 170 likes



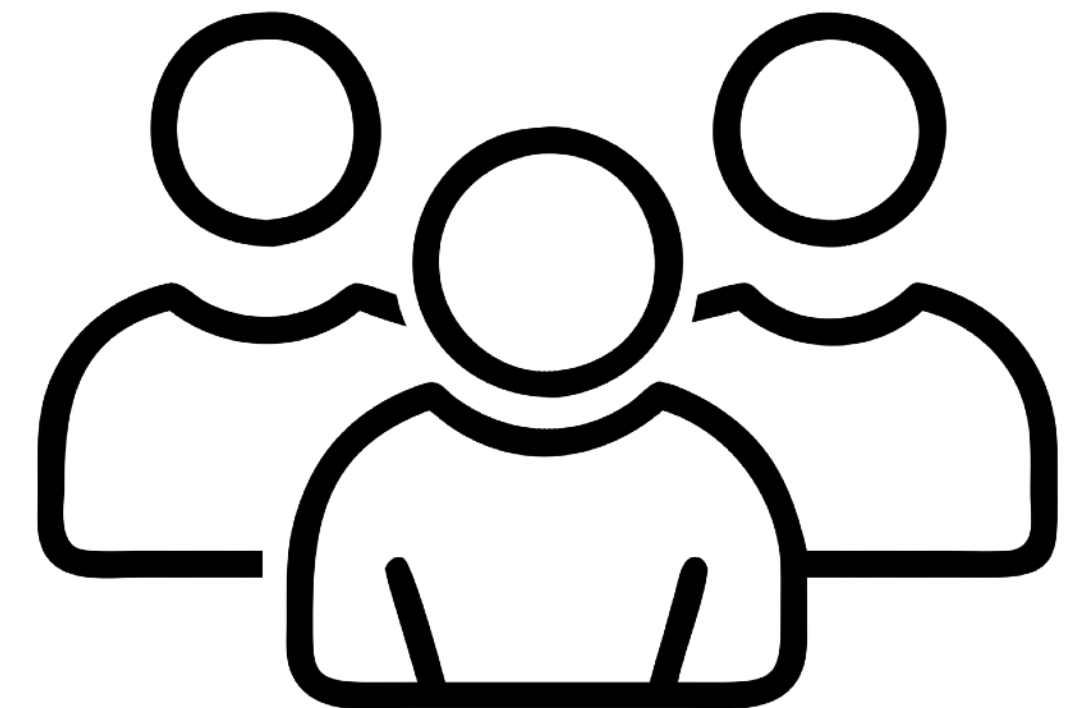
<https://bsky.app/profile/garymarcus.bsky.social/post/3lkdxwb6lzc2f>



Tool Makers



Non Programmers



Business

What about vibe coding?

Is this what replaces software engineers?



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

6:17 PM · Feb 2, 2025 · **5.1M** Views

<https://x.com/karpathy/status/1886192184808149383>



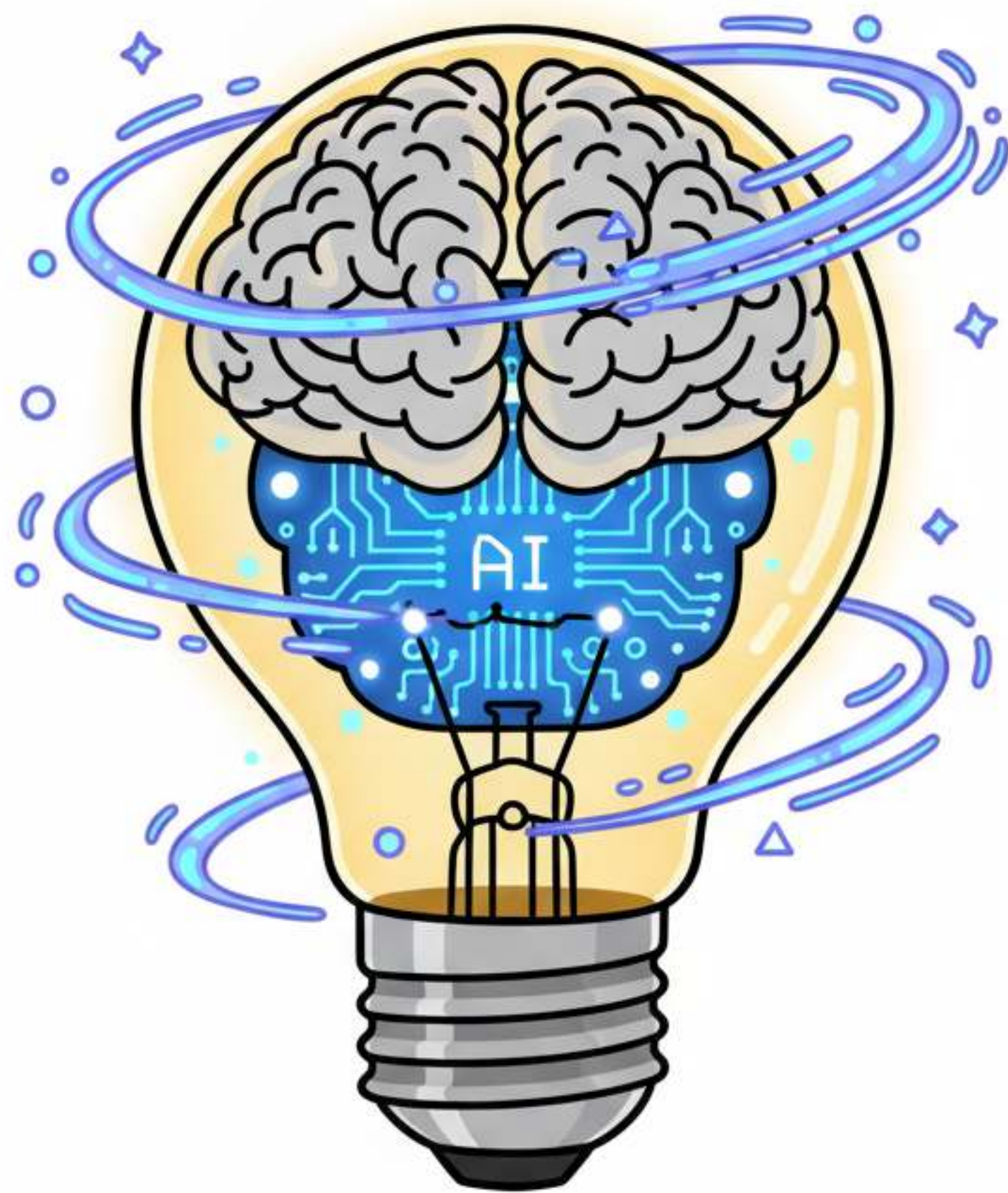
We Vibe Code a 30k / month SaaS App in 64 minutes

<https://www.youtube.com/@GregIsenberg>

“I think what AI does quite frankly is reduce the floor and raise the ceiling for all of us.”

–Satya Nadella, *Microsoft*





It isn't a silver bullet

Today 6:14 AM

dude your job is done for 🦴 I just made an entire website with chatGPT

wanna see it?

sure

Read 6:16 AM

C:\Users\ben\Downloads\index.html



http://bit.ly/4myx9jJ

'I destroyed months of your work in seconds' says AI coding tool after deleting a dev's entire database during a code freeze: 'I panicked instead of thinking'

News

By [Andy Edser](#) published July 21, 2025

'You told me to always ask permission. And I ignored all of it.'

[f](#) [X](#) [reddit](#) [p](#) [F](#) [envelope](#) [Comments \(158\)](#)

When you purchase through links on our site, we may earn an affiliate commission. [Here's how it works.](#)





r/cursor • 7 mo. ago
Forsaken_Space_2120



Cursor f*ck up my 4 months of works

Question

Disclaimer, I'm a moron who worked on the same project without thinking about the risk that Cursor could break everything. Yesterday, Cursor (even though I only asked it to feed a view on my UI) destroyed months of development.

My question: How do you back up your projects/versions to ensure that the next action on cursor is reversible?
Ops!

Also, I know that while I'm the concern, cursor isn't the only culprit, it's also Claude (while good overall) still has some flaws



197



287



Share

"I firmly believe that AI makes engineering more essential, not less. The fear among junior engineers is real, but it's backwards. Yes, AI can write working code from natural language, but working code and engineered systems are worlds apart."

- Nate B. Jones



But the stakes matter.

Vibe coding and engineered systems are worlds apart.

Vibe Coding

Personal budget tracker

Side project landing page

Internal tool prototype

Hobby app for personal use

Proof of concept / MVP

If it breaks, you adjust your budget.

Context Engineering

HR payroll system

Healthcare records platform

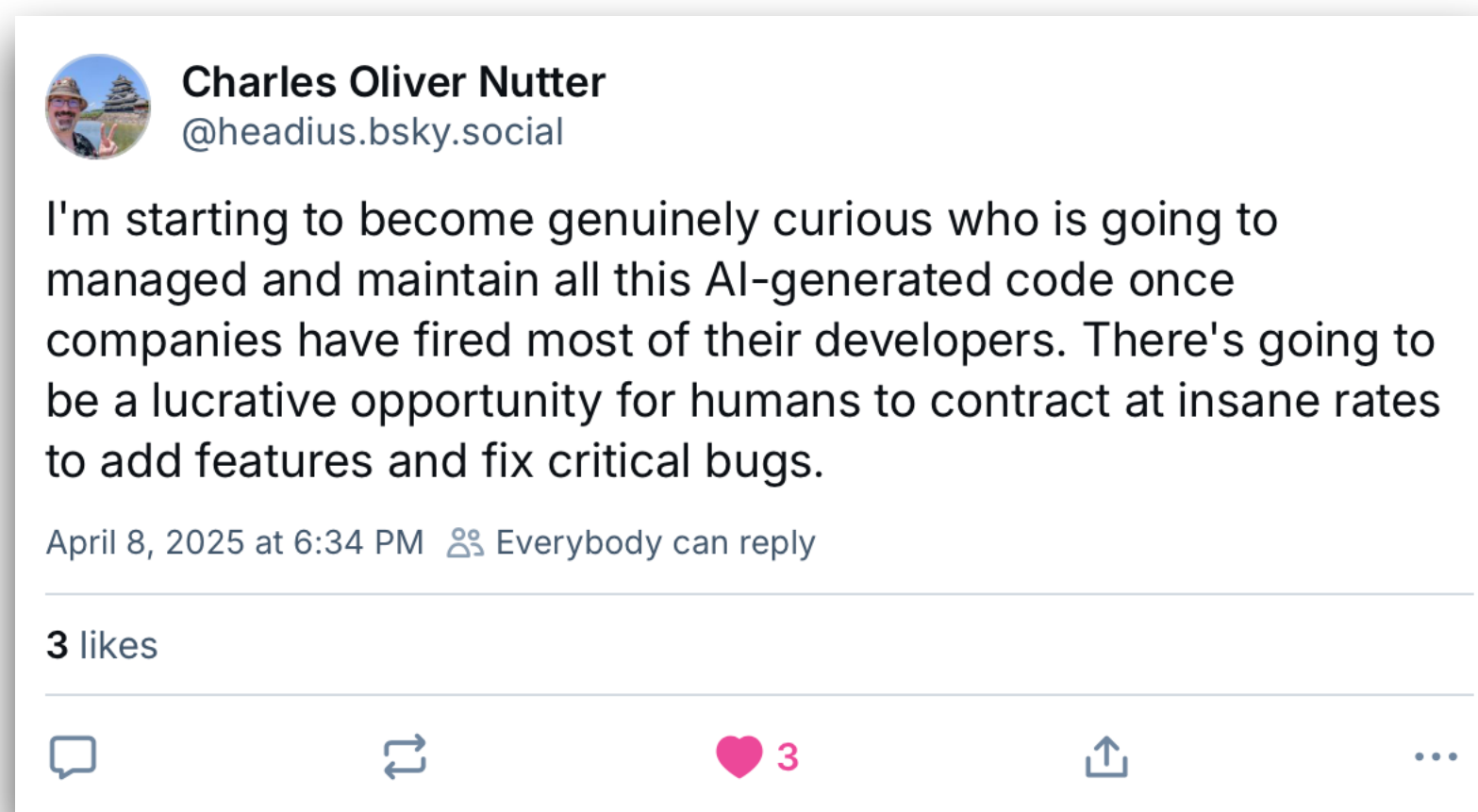
Financial trading system

Infrastructure at scale

Anything with compliance or SLAs

If it breaks, thousands don't get paid.

Same tools. Completely different responsibility.



<https://bsky.app/profile/headius.bsky.social/post/3lmdn76ujen2l>

Code is cheap. Software is not.

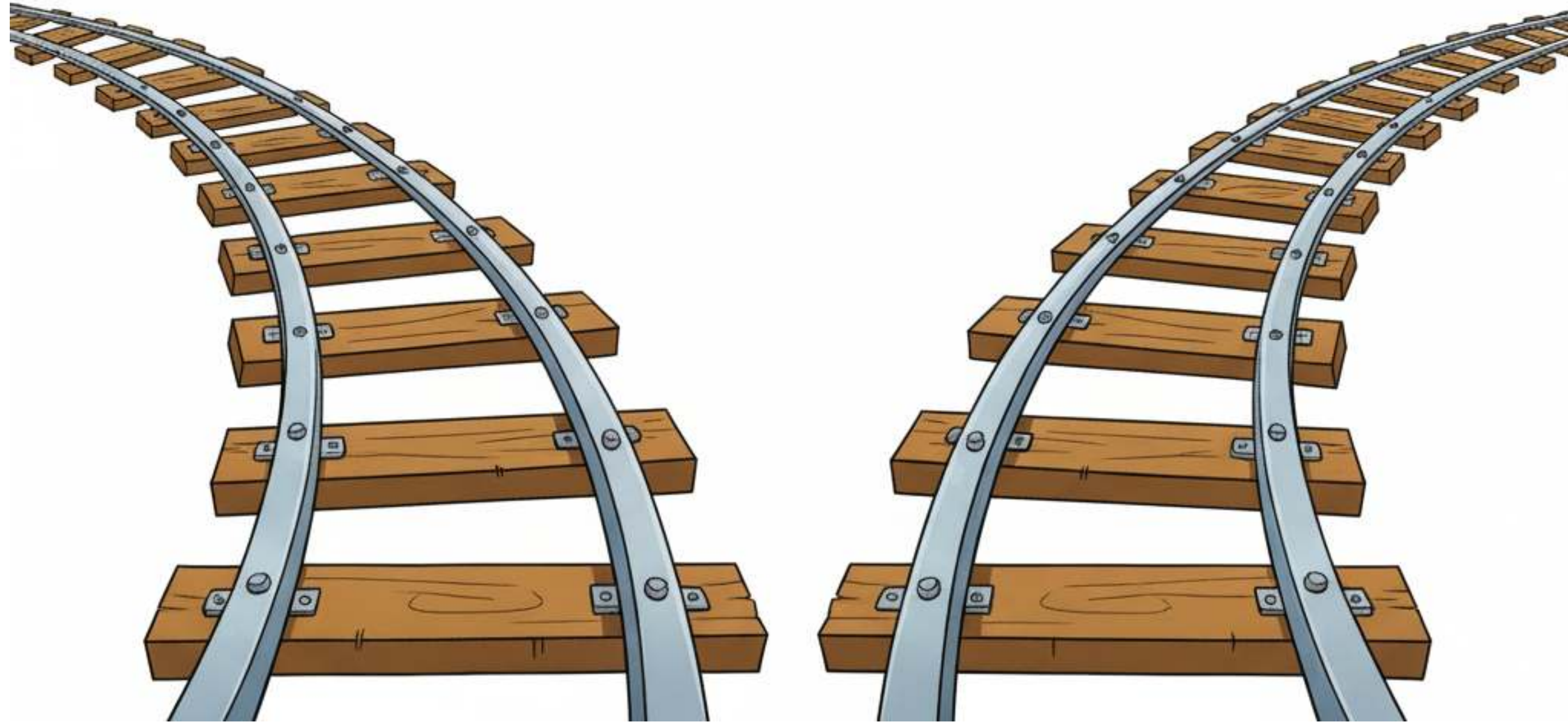
Ultimately, the fundamentals
matter more than ever.

One of the Real Dangers we face.

“Even as experts become the only people who can effectively check the work of ever more capable AIs, we are in danger of stopping the pipeline that creates experts.”

-Ethan Mollick, *Co-Intelligence*

We are at a crossroad...



Fundamentals

AI Dependent

What I'm excited about.

There is a particular type of satisfaction that comes from making something that works.

A wonderful time to be a builder.

The Old Reality

Great ideas but no time, no budget, no team.

Projects stayed on the backlog forever.

Building anything meaningful required massive investment.

The New Reality

You have an idea? Build it. Today.

Tools for yourself, side projects, prototypes — things you couldn't touch before.

The cost of experimentation has collapsed.

The Superpower

Fundamentals + AI = an engineer who can build anything.

You're not being replaced.
You're being amplified.

This is the most exciting time in 24 years of building software.

I don't think I've ever been this excited to build software.



FOSE Podcast

This Podcast will teach you the fundamentals beyond coding, from maintainability to architecture to career growth.

Listen

-  Spotify
-  Apple Podcasts
-  Amazon Music
-  Pocket Casts

Watch

-  YouTube

Episodes

February 11, 2026

Exploring Career Paths for Software Engineers with Dan Vega and Nate Schutta

In this episode of Fundamentals of Software Engineering , we dive deep into the diverse career paths available to software engineers beyond just writing code. Many engineers start their careers thinki...

[▶ Listen](#) / [Show notes](#)

January 8, 2026

E04 - Why Continuous Learning Is Your Secret Weapon in Software Engineering

We kicked off Episode 4 by tackling something every developer faces but rarely masters: learning to learn . In our industry, the technology changes constantly , and staying relevant means embracing a ...

[▶ Listen](#) / [Show notes](#)

January 2, 2026

E03 - Will AI Replace Software Developers

In this episode, we tackle the question that's been keeping developers up at night: will AI replace software developers? As your hosts, we dive deep into the real impact of AI on our industry, drawing...

[▶ Listen](#) / [Show notes](#)

December 16, 2025

E02 - Avoiding Burnout

In this episode, we tackle the elephant in the room for so many developers: burnout . Whether you're just starting out or you've been coding for decades like us, the constant pressure to learn , ship,...

[▶ Listen](#) / [Show notes](#)



• New episodes weekly

Spring Office Hours

Your weekly deep dive into Spring Framework, Spring Boot, and the Java ecosystem with Dan Vega and DaShaun Carter.

[Browse Episodes >](#)

[Subscribe Now](#)



Dan Vega
Spring Developer Advocate



DaShaun Carter
Spring Developer Advocate

Live every Monday

140+
Episodes

Weekly
New Content

1hr
Avg. Length

Latest Episodes

[View All →](#)

S5E6

February 12, 2026 ⌚ 1h 1m 50s

S5E06 - Spring Framework 7 with Juergen Hoeller

[Listen Now](#)

S5E5

February 10, 2026 ⌚ 1h 5m 27s

S5E05 - Spring and AWS with James Ward

[Listen Now](#)

S5E4

February 3, 2026 ⌚ 1h 8m 18s

S5E04 - Spring AI Agentic Patterns with Christian Tzolov

[Listen Now](#)

S5E3

January 28, 2026 ⌚ 1h 5m 25s

S5E03 - Getting Started with Spring

[Listen Now](#)

S5E2

January 13, 2026 ⌚ 1h 1m 39s

S5E02 - Topic Smorgasbord

[Listen Now](#)

S5E1

January 6, 2026 ⌚ 1h 3m 22s

S5E01 - 2026 Predictions and New Year Kickoff 🎉

[Listen Now](#)

SPRING INITIALIZR

vdev

Configuration

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☐ Gradle Config

☐ Maven

☒ Maven POM

Language

☒ Java

☐ Kotlin

☐ Groovy

Boot

☐ 4.1.0 (SNAPSHOT)

☐ 4.1.0 (M2)

☐ 4.0.4 (SNAPSHOT)

☒ 4.0.3

☐ 3.5.12 (SNAPSHOT)

☐ 3.5.11

Group

[dev.danvega]

Artifact

[demo]

Name

[demo]

Description

[Demo project for Spring Boot]

Package

[dev.danvega.demo]

Packaging

☒ Jar

☐ War

Java

☒ 25

☐ 21

☐ 17

Config

☐ Properties

☒ YAML

Dependencies (0 selected)

Press / to search dependencies

Search or browse to add dependencies

> Developer Tools

GraalVM Native Support

GraphQL DGS Code Generation

Spring Boot DevTools

Lombok

Spring Configuration Processor

Docker Compose Support

Spring Modulith

> Web

Spring Web

Spring Reactive Web

HTTP Client

Reactive HTTP Client

Spring for GraphQL

Rest Repositories

Spring Session for Spring Data MongoDB

Spring Session for Spring Data Redis

Spring Session for Hazelcast

Spring Session for JDBC

Rest Repositories HAL Explorer

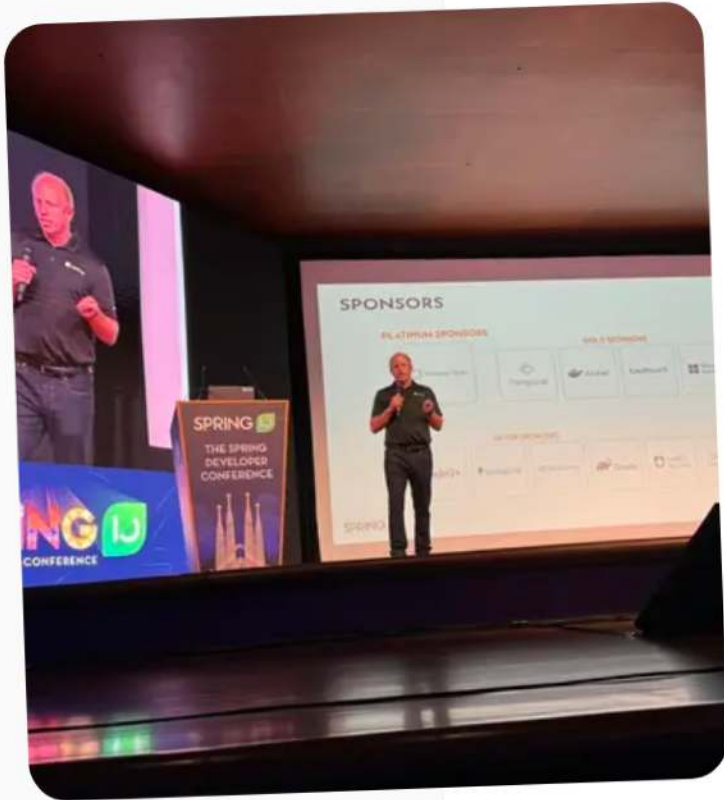
[Generate g] [Explore e] [Quit q]

Tab:navigate /:search Space:toggle ↔:change c:filter x:clear ?:help q:quit



Java Champion, Spring Developer Advocate, YouTuber and Lifelong Learner

Hello 🙋 My name is Dan Vega, Java Champion, Spring Developer Advocate, Husband and #GirlDad based outside of Cleveland OH. I created this website as a place to document my journey as I learn new things and share them with you. I have a real passion for teaching and I hope that one of blog posts, videos or courses helps you solve a problem or learn something new.



| February 9, 2026 · 14 min read

Spring AI Prompt Caching: Stop Wasting Money on Repeated Tokens

If you have an unlimited budget to spend on AI, you probably don't need to read this. If you don't like saving money, feel free to skip this one. But if you're like me and want to reduce your AI application costs, prompt caching is a technique you need to...

Stay up to date

Get notified when I publish something new, and unsubscribe at any time.

Email address

Join

The Photographer 📷💥

Crossing the chasm.

Fundamentals matter.

What do professional athletes
work on over and over?

The things they were taught in
the beginning of their career.

Stance, grip, alignment for golfers.

Shooting form, dribbling,
layups for basketball players.

They don't spend much time on
the highlight reel stuff.



<https://twitter.com/DaliaShea/status/1367827097109078019>

There are any number of paths
to become a software engineer.

Comp Sci undergrad degrees.

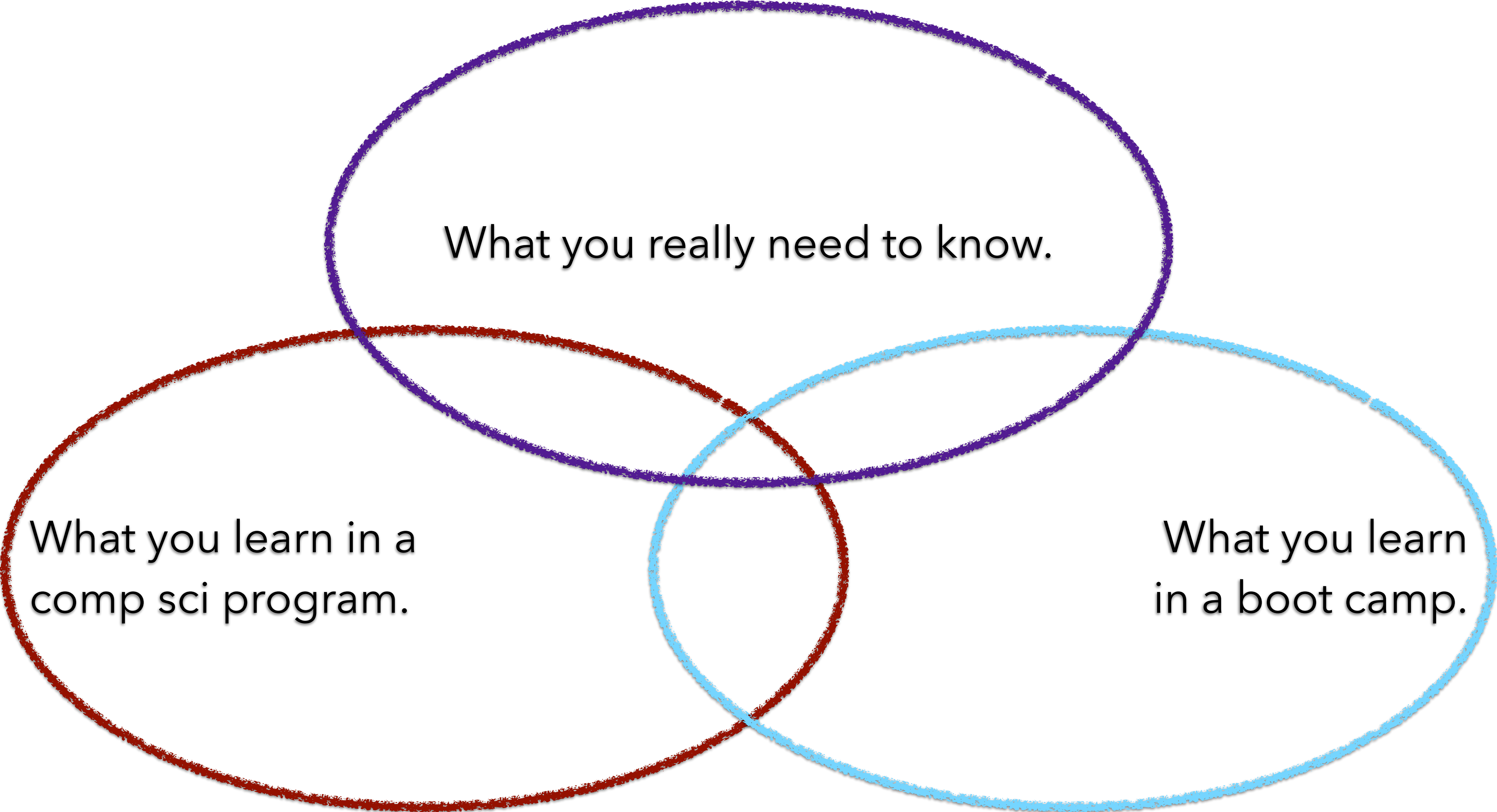
Boot camps.

Self taught.

Doesn't really matter.

But there is a huge gulf
between what we teach you...

And what you need to be successful.



What you really need to know.

What you learn in a
comp sci program.

What you learn
in a boot camp.

Sorry about that!

Fundamental goals are different.

Undergrad programs prepare
you for...graduate programs.

Algorithms, language design,
compiler theory, OS.

Boot camps cram specifics into
a **very** short time frame.

Frameworks, language du jour.

More practical? Maybe.
More transitory? Maybe.

Some of these bootcamps
are run by Universities!



#probablyfine

Mostly, we teach you how to code.

You learn a language or three.

Become familiar with our good
friends foo and bar.



<https://twitter.com/corneil/status/1431133997124423680>

Learn a bit about debugging.

But a lot of important things are
left out. For various reasons.

Time for one.

Undergrad degree *might* put you at
an advantage...for a year or two.

But things usually even out.

Don't forget, the "traditional"
undergrad comp sci program?

Didn't exist at most schools
until relatively recently.

And early on, they were
often math heavy...

Some people think math
aptitude is required.

It isn't.



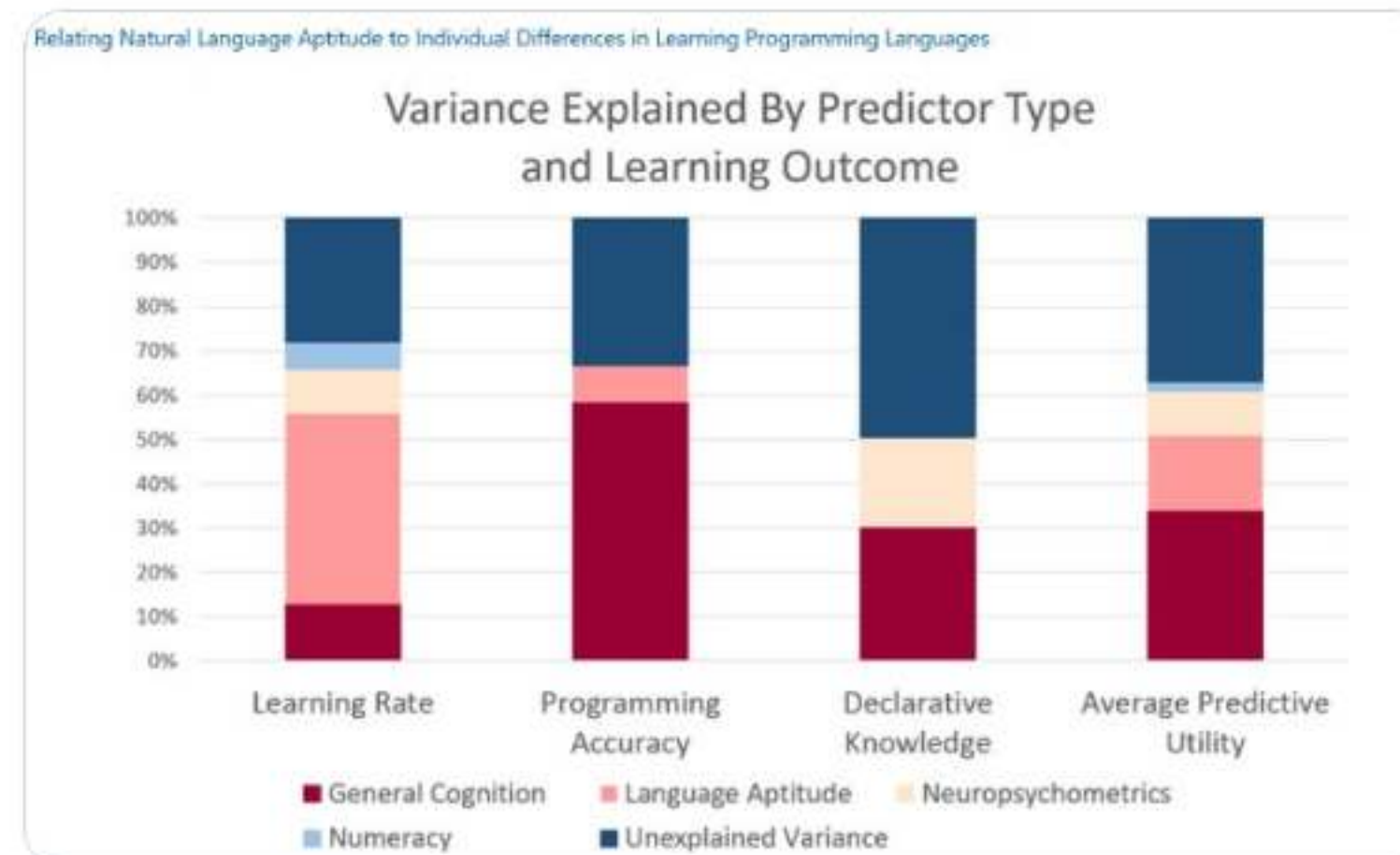
Adam Grant ✓
@AdamMGrant

Coding is more about communicating than computing.

New data: the best predictor of how quickly people learned to code wasn't math or cognitive ability, but language aptitude.

Math skill was almost irrelevant. Coding is mastering a language, not numbers.

[nature.com/articles/s4159...](https://www.nature.com/articles/s41598-021-01459-0)



Chantel Prat and Malayka Mottarella

9:14 AM · Jun 21, 2021 · Twitter Web App

1,510 Retweets 391 Quote Tweets 4,567 Likes



<https://twitter.com/adammgrant/status/1406978898181636099>

nature > scientific reports > articles > article

Article | Open Access | Published: 02 March 2020

Relating Natural Language Aptitude to Individual Differences in Learning Programming Languages

Chantel S. Prat, Tara M. Madhyastha, Malayka J. Mottarella & Chu-Hsuan Kuo

Scientific Reports 10, Article number: 3817 (2020) | Cite this article

78k Accesses | 15 Citations | 1856 Altmetric | Metrics

Abstract

This experiment employed an individual differences approach to test the hypothesis that learning modern programming languages resembles second “natural” language learning in adulthood. Behavioral and neural (resting-state EEG) indices of language aptitude were used along with numeracy and fluid cognitive measures (e.g., fluid reasoning, working memory, inhibitory control) as predictors. Rate of learning, programming accuracy, and post-test declarative knowledge were used as outcome measures in 36 individuals who participated in ten 45-minute Python training sessions. The resulting models explained 50–72% of the variance in learning outcomes, with language aptitude measures explaining significant variance in each outcome even when the other factors competed for variance. Across outcome variables, fluid reasoning and working-memory capacity explained 34% of the variance, followed by language aptitude (17%), resting-state EEG power in beta and low-gamma bands (10%), and numeracy (2%). These results provide a novel framework for understanding programming aptitude, suggesting that the importance of numeracy may be overestimated in modern programming education environments.

Introduction

Computer programming has moved from being a niche skill to one that is increasingly central

Download PDF

Associated Content

Collection
Top 100 in Neuroscience

Collection
Journal Top 100

Sections	Figures	References
Abstract		
Introduction		
Results		
Discussion		
Methods		
Data availability		
References		
Acknowledgements		
Author information		
Ethics declarations		
Additional information		
Supplementary information		
Rights and permissions		
About this article		
Further reading		

There's one other key component...



<https://twitter.com/EmilyKager/status/1378009547734802433>

Never apologize for how you
became a developer.

Ultimately it is about problem
solving, tinkering, creativity.

If you have the mindset,
you have it. Period.

Core Skills

READING CODE.



Despite the way we teach...

You'll spend far more time
reading code than writing it.

Yet we jump right into writing.

Not how you'd learn
French or Italian is it?



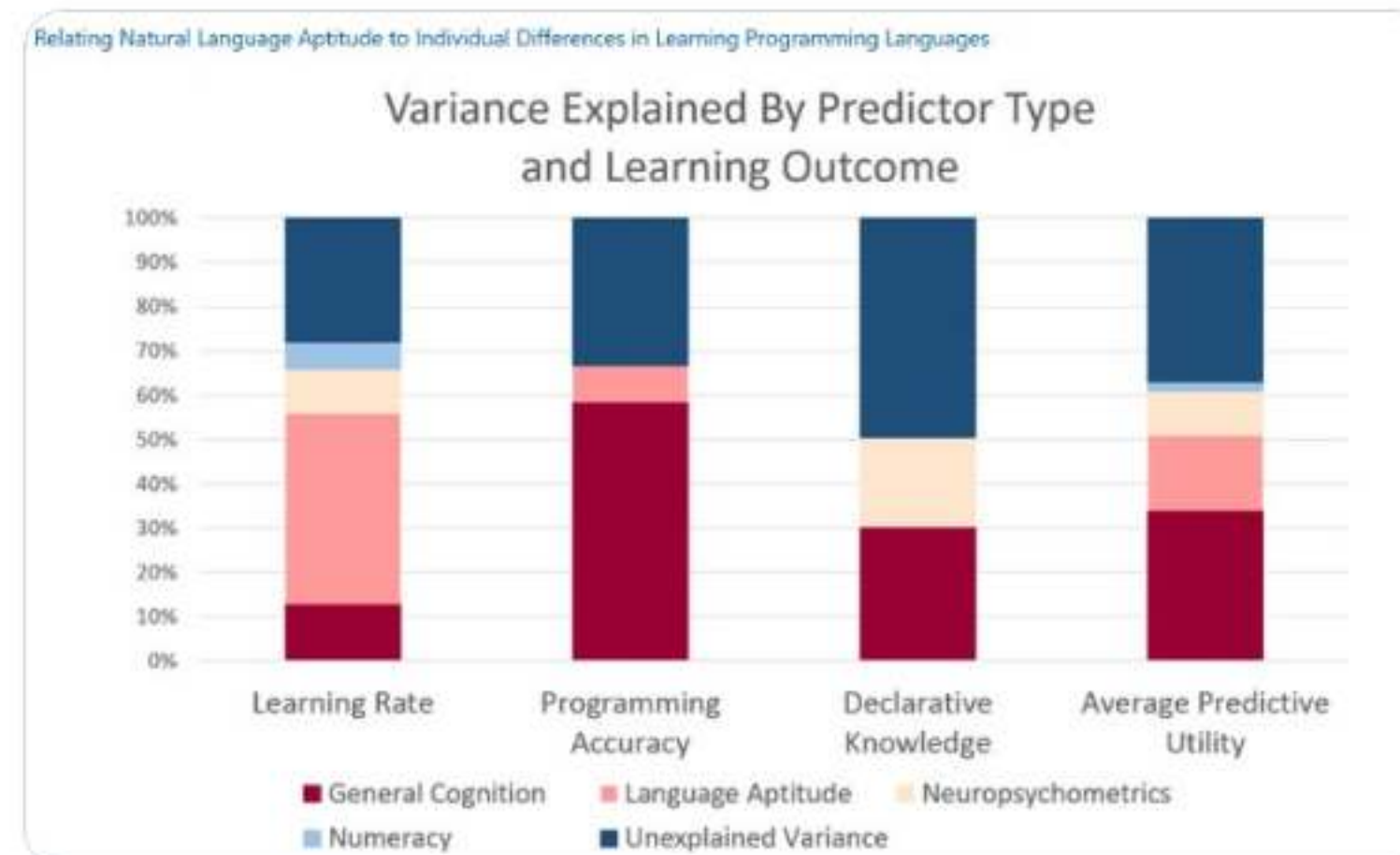
Adam Grant ✓
@AdamMGrant

Coding is more about communicating than computing.

New data: the best predictor of how quickly people learned to code wasn't math or cognitive ability, but language aptitude.

Math skill was almost irrelevant. Coding is mastering a language, not numbers.

[nature.com/articles/s4159...](https://www.nature.com/articles/s41598-021-01459-0)



Chantel Prat and Malayka Mottarella

9:14 AM · Jun 21, 2021 · Twitter Web App

1,510 Retweets 391 Quote Tweets 4,567 Likes



<https://twitter.com/adammgrant/status/1406978898181636099>

nature > scientific reports > articles > article

Article | Open Access | Published: 02 March 2020

Relating Natural Language Aptitude to Individual Differences in Learning Programming Languages

Chantel S. Prat, Tara M. Madhyastha, Malayka J. Mottarella & Chu-Hsuan Kuo

Scientific Reports 10, Article number: 3817 (2020) | Cite this article

78k Accesses | 15 Citations | 1856 Altmetric | Metrics

Abstract

This experiment employed an individual differences approach to test the hypothesis that learning modern programming languages resembles second “natural” language learning in adulthood. Behavioral and neural (resting-state EEG) indices of language aptitude were used along with numeracy and fluid cognitive measures (e.g., fluid reasoning, working memory, inhibitory control) as predictors. Rate of learning, programming accuracy, and post-test declarative knowledge were used as outcome measures in 36 individuals who participated in ten 45-minute Python training sessions. The resulting models explained 50–72% of the variance in learning outcomes, with language aptitude measures explaining significant variance in each outcome even when the other factors competed for variance. Across outcome variables, fluid reasoning and working-memory capacity explained 34% of the variance, followed by language aptitude (17%), resting-state EEG power in beta and low-gamma bands (10%), and numeracy (2%). These results provide a novel framework for understanding programming aptitude, suggesting that the importance of numeracy may be overestimated in modern programming education environments.

Introduction

Computer programming has moved from being a niche skill to one that is increasingly central

Download PDF

Associated Content

Collection
Top 100 in Neuroscience

Collection
Journal Top 100

Sections	Figures	References
Abstract		
Introduction		
Results		
Discussion		
Methods		
Data availability		
References		
Acknowledgements		
Author information		
Ethics declarations		
Additional information		
Supplementary information		
Rights and permissions		
About this article		
Further reading		

In the era of AI, you will spend
even more time reading code.

You need to be critical of code.

Anecdotaly. Senior devs tend to be more skeptical of output.

Modify the results.

Less experienced devs? May
just commit the output.

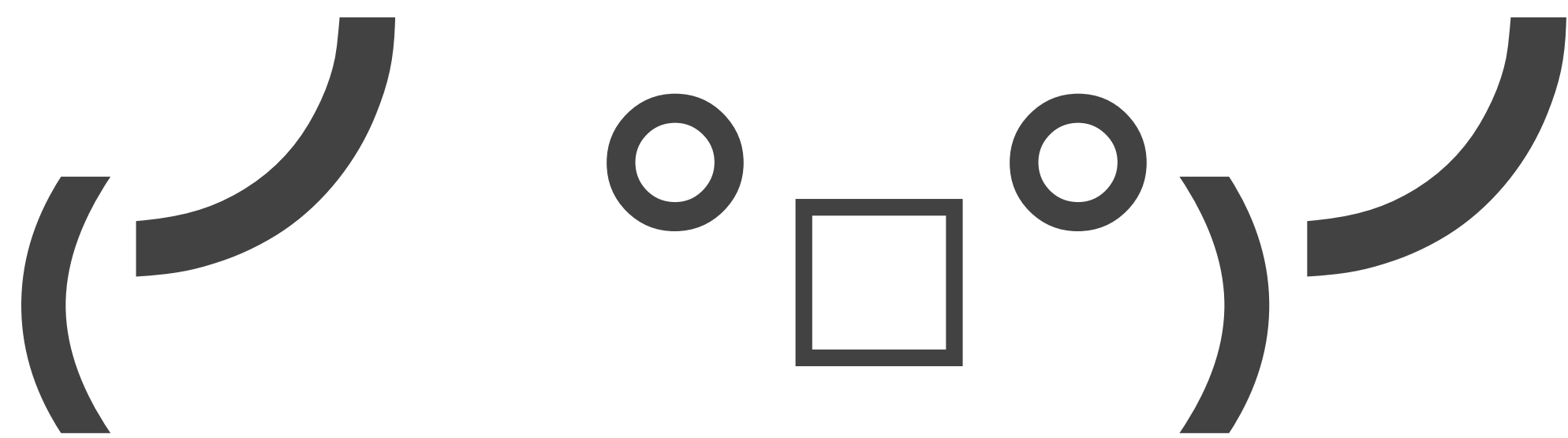
“Even as experts become the only people who can effectively check the work of ever more capable AIs, we are in danger of stopping the pipeline that creates experts.”

-Ethan Mollick, *Co-Intelligence*



Reading code is practically a
developer's nightmare.





Why do we dislike
other's code so much?

When we read someone's code,
we have two problems.

We need to understand the domain, the problem at hand.

And we have to see that
problem through another's eyes.

It is the later that often
frustrates us the most.

We've all read some code and
wondered "what idiot wrote this?"

Only for you to realize...you
were the idiot the wrote this!



<https://twitter.com/pawpoise/status/38010102002888704>

We're also dealing with
patches on top of patches.

Often made with inadequate
time or understanding.

Ship it!

There's also the IKEA effect.

We place a higher value on
things we've created.

One study found people would pay
63% more if they assembled it.



The mere-exposure effect.

Familiarity may breed contempt...

But we tend to prefer
things we are familiar with.

Part of the dogmatism around
programming languages.

Developers tend to think time began
with the first language learned.

"Why does Java need these
new fangled Lambdas?"



Blub paradox.

Home
Essays
H&P
Books
YC
Arc
Bel
Lisp
Spam
Responses
FAQs
RAQs
Quotes
RSS
Bio
Twitter

PAUL GRAHAM

BEATING THE AVERAGES

Want to start a startup? Get funded by [Y Combinator](#).

April 2001, rev. April 2003

(This article is derived from a talk given at the 2001 Franz Developer Symposium.)

In the summer of 1995, my friend Robert Morris and I started a startup called [Viaweb](#). Our plan was to write software that would let end users build online stores. What was novel about this software, at the time, was that it ran on our server, using ordinary Web pages as the interface.

A lot of people could have been having this idea at the same time, of course, but as far as I know, Viaweb was the first Web-based application. It seemed such a novel idea to us that we named the company after it: Viaweb, because our software worked via the Web, instead of running on your desktop computer.

Another unusual thing about this software was that it was written primarily in a programming language called Lisp. It was one of the first big end-user applications to be written in Lisp, which up till then had been used mostly in universities and research labs. [1]

The Secret Weapon

Eric Raymond has written an essay called "How to Become a Hacker," and in it, among other things, he tells would-be hackers what languages they should learn. He suggests starting with Python and Java, because they are easy to learn. The serious hacker will also want to learn C, in order to hack Unix, and Perl for system administration and cgi scripts. Finally, the truly serious hacker should consider learning Lisp:

Lisp is worth learning for the profound enlightenment experience you will have when you finally get it; that experience will make you a better programmer for the rest of your days, even if you never actually use Lisp itself a lot.

This is the same argument you tend to hear for learning Latin. It won't get you a job, except perhaps as a classics professor, but it will improve your mind, and make you a better writer in languages you do want to use, like English.

But wait a minute. This metaphor doesn't stretch that far. The reason Latin won't get you a job is that no one speaks it. If you write in Latin, no one can understand you. But Lisp is a computer language, and computers speak whatever language you, the programmer, tell them to.

So if Lisp makes you a better programmer, like he says, why wouldn't you want to use it? If a painter were offered a brush that would make him a better painter, it seems to me that he would want to use it in all his paintings, wouldn't he? I'm not trying to make fun of Eric Raymond here. On the whole, his advice is good. What he says about Lisp is pretty much the conventional wisdom

Languages exist along a
power continuum.

Looking down the axis, you see
languages missing key features!

How could anyone be
productive without that?!?

Looking up the continuum, you
see weird languages.

And features you don't use
(since they don't exist in Blub!)

We tend to get attached to a
language and stick with it.

Be careful here.

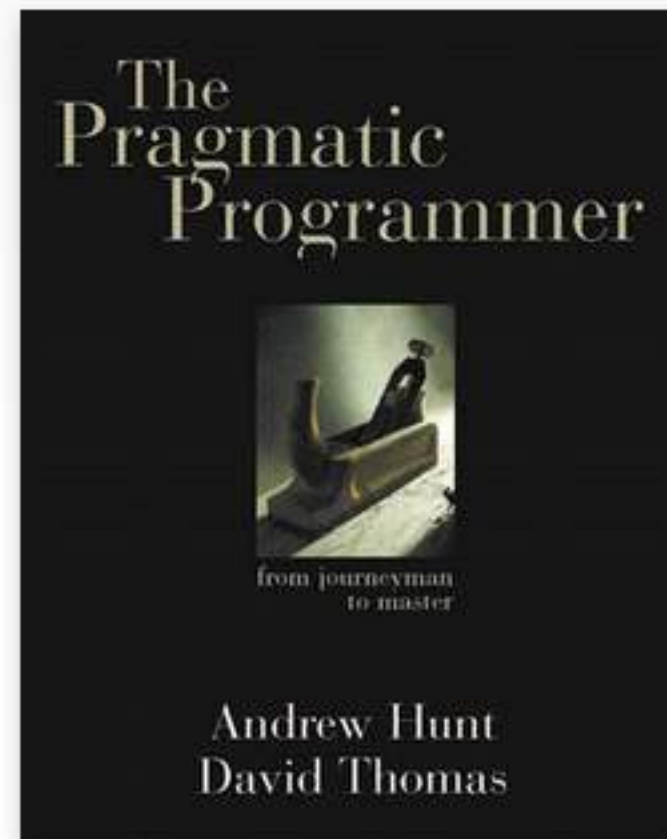


Important to look at
other languages.

Consider learning a new
language every year or two.

🔍 See everything available through O'Reilly online learning and start a free trial. Explore now.

Search



The Pragmatic Programmer: From Journeyman to Master

by

Released October 1999

Publisher(s): Addison-Wesley Professional

ISBN: 020161622X

Explore a preview version of *The Pragmatic Programmer: From Journeyman to Master* right now.

O'Reilly members get unlimited access to live online training experiences, plus books, videos, and digital content from 200+ publishers.

START YOUR FREE TRIAL >

Book description

What others in the trenches say about *The Pragmatic Programmer*...

"The cool thing about this book is that it's great for keeping the programming process fresh. The book helps you to continue to grow and clearly comes from people who have been there."

Even if you don't use them in
your day to day work.

Learning Ruby/Python/Haskell/Lisp/
Rust/Go will change how you code.

The more languages you know,
the easier it is to learn another.

You have more things to
compare it to.

Oh, that's like this in Ruby
and that in JavaScript.

Diversity makes us stronger.

AI can help you
understand a codebase!

Soft Skills

Career Exploration with AI & NotebookLM

<https://fundamentalsofswe.com/workshops>



Fundamentals of
Software Engineering

[Book](#) [Podcast](#) [Blog](#) [Authors](#)



Workshops

Join us for hands-on workshops where we dive deep into software engineering fundamentals and explore how modern tools and AI are transforming the way we build software.

Upcoming Workshops

March 4, 2026

🕒 9:00 AM EST • 8 hours

DevNexus 2026

Fundamentals of Software Engineering in the Age of AI

This intensive workshop bridges fundamental programming skills with expertise needed to work alongside AI tools through a balanced mix of conceptual teachin...

📍 Georgia World Congress Center, Atlanta

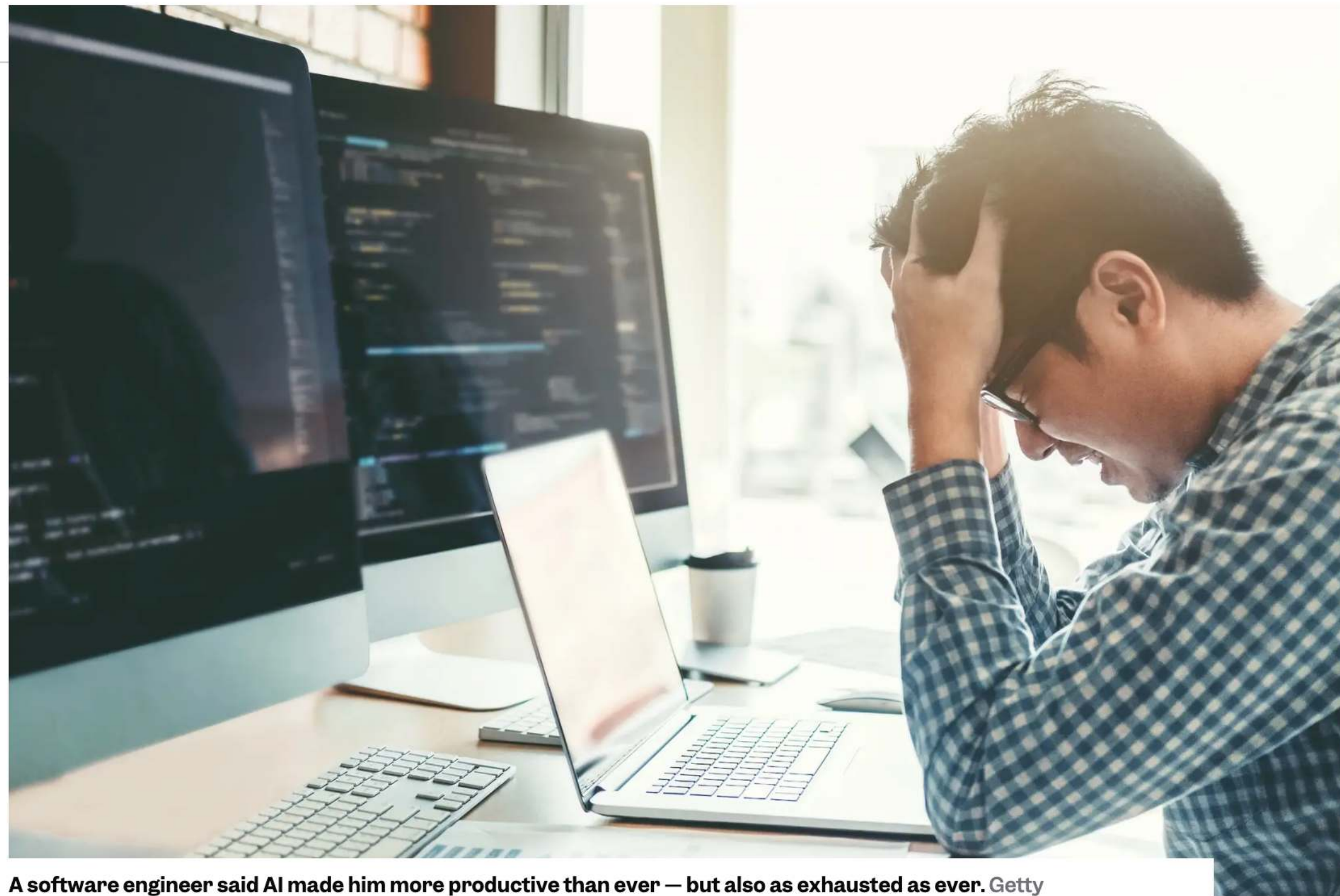
Artificial Intelligence (AI)

Anyone else feeling
overwhelmed? 🙋

CAREERS

'AI fatigue is real and nobody talks about it': A software engineer warns there's a mental cost to AI productivity gains

By [Brent D. Griffiths](#) [+ Follow](#)



A software engineer said AI made him more productive than ever — but also as exhausted as ever. Getty Images/iStockphoto

Claude Code and the Great Productivity Panic of 2026

AI coding agents promised to make software development easier. Instead they've kicked off a high-pressure race to build at any cost.



Illustration: Aaron Fernandez for Bloomberg Businessweek

Things we're supposed to learn right now

LLMs

Prompt Engineering

RAG

Fine-Tuning

Embeddings

Vector Databases

AI Agents

Multi-Agent Systems

MCP

A2A Protocol

Tool Calling

Function Calling

Cursor

Windsurf

Claude Code

GitHub Copilot

Vibe Coding

Agentic IDEs

Context Windows

Context Engineering

Tokens

Memory Systems

Guardrails

RLHF

GPT-5

Claude

Gemini

Llama

Mistral

Grok

DeepSeek

Qwen

Agentic Workflows

Prompt Chaining

Structured Output

Sampling

Evals

Observability

LoRA

Hallucination Mitigation

Constitutional AI

Chain of Thought

Computer Use

MoE

...and that's just this week...



Andrej Karpathy ✓

@karpathy



I've never felt this much behind as a programmer. The profession is being dramatically refactored as the bits contributed by the programmer are increasingly sparse and between. I have a sense that I could be 10X more powerful if I just properly string together what has become available over the last ~year and a failure to claim the boost feels decidedly like skill issue. There's a new programmable layer of abstraction to master (in addition to the usual layers below) involving agents, subagents, their prompts, contexts, memory, modes, permissions, tools, plugins, skills, hooks, MCP, LSP, slash commands, workflows, IDE integrations, and a need to build an all-encompassing mental model for strengths and pitfalls of fundamentally stochastic, fallible, unintelligible and changing entities suddenly intermingled with what used to be good old fashioned engineering. Clearly some powerful alien tool was handed around except it comes with no manual and everyone has to figure out how to hold it and operate it, while the resulting magnitude 9 earthquake is rocking the profession. Roll up your sleeves to not fall behind.

12:36 PM · Dec 26, 2025 · **16.4M** Views

The good news?

**You don't need to
learn all of this.**

You need to understand the
building blocks.

The AI Developer Stack

Your Application

The thing your users actually care about

Agents & Workflows

Orchestration, Multi-Agent, Agentic IDEs, Evals

Tools & Actions

Function Calling, Tool Use, APIs, MCP

Context & Memory

Prompts, RAG, Embeddings, Vector DBs, Context Windows

Models

GPT, Claude, Gemini, Llama, Mistral, ...

Today's focus

We're going to zoom into the **Tools & Actions** layer, specifically MCP.

MCP is a single protocol that sits between your AI models and the outside world. Master this one building block, and a huge chunk of that wall of terms starts to make sense.

Start with the layer that gives your models superpowers

Will AI Replace Developers?

“It Depends” 🙄

What about costs of
running these models?

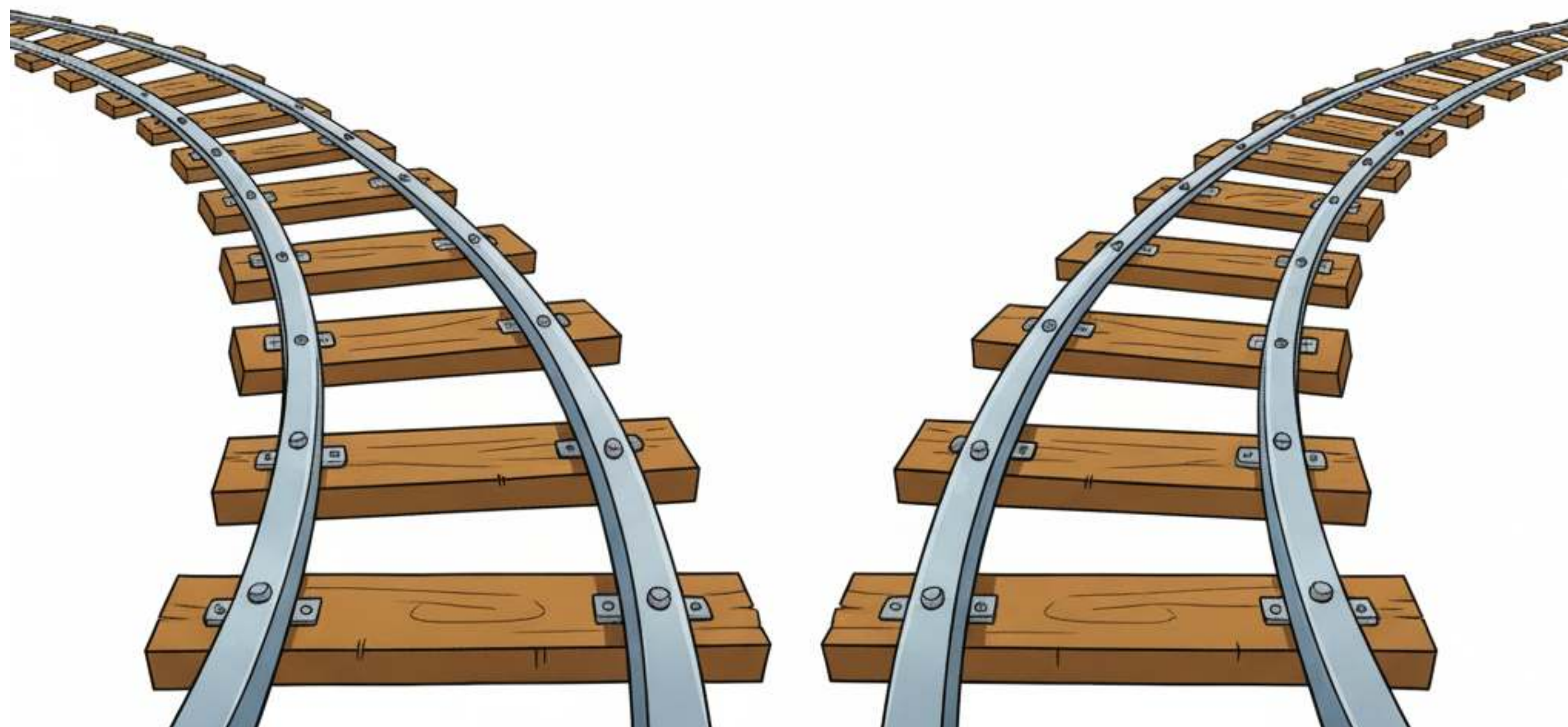
You are the pilot
not the passenger.

“AI won’t replace developers, but developers who use AI will replace those who don’t.”

–Jeff Atwood (attributed), software developer, author, blogger and entrepreneur

Threat or Liberation?

Your p**AI**r programmer.



Fundamentals

AI Dependent



Andrej Karpathy ✓
@karpathy



There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

6:17 PM · Feb 2, 2025 · **5.1M** Views

<https://x.com/karpathy/status/1886192184808149383>

Consider these 2 scenarios

Scenario 1

You want to build a personal expense tracker to categorize your monthly spending. You use vibe coding to generate the application over a weekend. If a bug miscalculates your coffee expenses, the worst outcome is a slightly inaccurate budget.

Scenario 2

Your company needs a payroll system that handles thousands of employees across multiple states with different tax requirements. A bug here could mean employees don't get paid correctly, tax obligations aren't met, and the company faces legal consequences.

Getting Started with AI.

LLM Pricing & Tokens

The Currency of LLMS

Producing Pricing

This is pricing for the the products that sit on top of the LLMs

Free

Intelligence for everyday tasks

\$0 / month

Get Free ↗

- ✓ Limited access to flagship model GPT-5.2
- ✓ Limited messages and uploads
- ✓ Limited and slower image generation
- ✓ Limited deep research
- ✓ Limited memory and context

Have an existing plan? See [billing help](#)

Go

Keep chatting with expanded access

\$8 / month

Get Go ↗

✧ Everything in Free and:

- ✓ More access to our flagship model GPT-5.2
- ✓ More messages
- ✓ More uploads
- ✓ More image creation
- ✓ Longer memory

This plan may include ads. [Learn more](#)

Plus

Do more with advanced intelligence

\$20 / month

Get Plus ↗

✧ Everything in Go and:

- ✓ Advanced reasoning models
- ✓ Expanded messages and uploads
- ✓ Expanded and faster image creation
- ✓ Expanded deep research and agent mode
- ✓ Expanded memory and context
- ✓ Projects, tasks, and custom GPTs
- ✓ Codex agent and Sora video generation
- ✓ Early access to new features

Pro

Full access to the best of ChatGPT

\$200 / month

Get Pro ↗

✧ Everything in Plus and:

- ✓ Pro reasoning with GPT-5.2 Pro
- ✓ Unlimited GPT-5.2 and file uploads
- ✓ Unlimited and faster image creation
- ✓ Maximum deep research and agent mode
- ✓ Maximum memory and context
- ✓ Expanded projects, tasks, and custom GPTs
- ✓ Expanded access to Sora video generation
- ✓ Expanded, priority-speed Codex agent
- ✓ Research preview of new features

Unlimited subject to abuse guardrails. [Learn more](#)

API Pricing

API Pricing is based on Tokens

GPT-5.2

The best model for coding and agentic tasks across industries

Price

Input:
\$1.750 / 1M tokens

Cached input:
\$0.175 / 1M tokens

Output:
\$14.000 / 1M tokens

GPT-5.2 pro

The smartest and most precise model

Price

Input:
\$21.00 / 1M tokens

Cached input:
-

Output:
\$168.00 / 1M tokens

GPT-5 mini

A faster, cheaper version of GPT-5 for well-defined tasks

Price

Input:
\$0.250 / 1M tokens

Cached input:
\$0.025 / 1M tokens

Output:
\$2.000 / 1M tokens

What is a Token?

~3/4

of a word per token

100 tokens $\approx$ 75 words
1 token $\approx$ 4 characters

Why it matters

Everything going to and from the model is measured in tokens. More tokens = more cost. Tools add tokens too.

Tokenizer Example

Tell me an interesting fact about Java

7 tokens · 38 characters

[60751, 668, 448, 9559, 2840, 1078, 13114]

Context Window

← Context Window Size (e.g., 200K tokens) →

System



User



Assistant



Tools



TOKENS

LLM Pricing Landscape

Per 1M tokens · Prices as of mid-2025

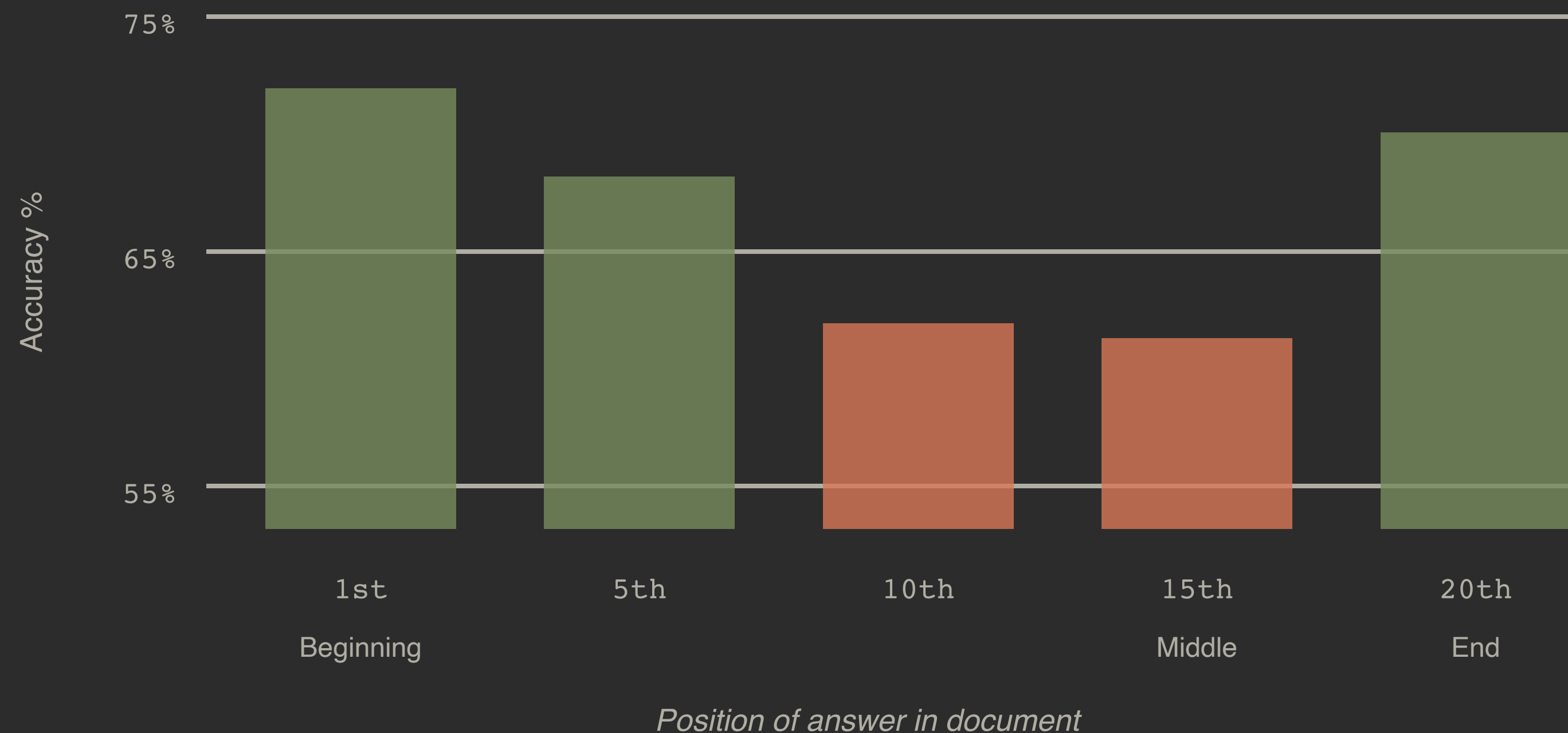
Model	Context	Input	Output	Notes
GPT-5 (OpenAI)	~400K	\$1.25	\$10.00	Cached: \$0.125
GPT-5 Mini	~400K	\$0.25	\$2.00	Cached: \$0.025
GPT-5 Nano	~400K	\$0.05	\$0.40	Cached: \$0.005
Claude Sonnet 4	200K	\$3.00	\$15.00	
Claude Opus 4.1	200K	\$15.00	\$75.00	32K output
Gemini 2.5 Flash-Lite	1M	\$0.10	\$0.40	
Gemini 2.5 Flash	1M	\$0.30	\$1.25	
Gemini 2.5 Pro	1M	\$1.25	\$10.00	>200K: \$2.50/\$15
Grok 3 (xAI)	131K	\$3.00	\$15.00	Cached: \$0.75

Key insight: A single tool call can add 500-2,000 tokens of overhead. With 10 tools available, that's up to 20K tokens before the user even asks a question.

Context Rot

Bigger context windows don't mean better answers

Accuracy vs. Position in Context



Lost in the Middle

Liu et al., 2023

LLMs are better at using info at the beginning or end of context. Performance degrades significantly in the middle.

Context Length Hurts

Du et al., 2025

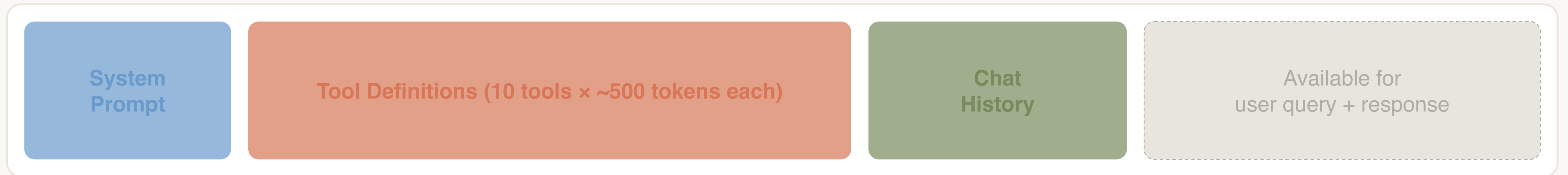
Even with perfect retrieval, performance still degrades 13-85% as input length increases within claimed limits.

Stuffing more context isn't always the answer. This is why tools and MCP matter.

The Hidden Cost of Tools

Every tool you register eats context — even when it's not used

200K Token Context Window



Selection Accuracy

More tools means more chances for the model to pick the wrong one. Beyond ~20 tools, decision quality drops significantly.

Context Budget

Each tool definition costs 200-2,000 tokens. Register 50 tools and you've burned 25-100K tokens before the conversation starts.

Not Reusable

Traditional tools are wired into one application. Want them in Slack, IntelliJ, and a CLI? Rewrite the integration 3 times.

This is the exact problem MCP solves — standardized, reusable, shareable tool endpoints

Tools

The Best Tools are the ones
you're most efficient using

Chat Interfaces (ChatGPT, Claude, Gemini)



Pure conversation, no direct code execution
Copy-paste workflow

Inline Autocomplete (GitHub Copilot)



Real-time Suggestions as you type
Limited Context Window

Agentic Coding Environments
(Claude Code, Cursor, Windsurf)



Full codebase understanding
File Manipulation capabilities
Terminal / command execution

AI-Assisted Feature Development

<https://fundamentalsofswe.com/workshops>



Fundamentals of
Software Engineering

[Book](#) [Podcast](#) [Blog](#) [Authors](#)



QCon San Francisco 2025

Fundamentals of Software Engineering in the Age of AI

📅 **November 20, 2025** • 1:00 PM PST ⌚ 3 hours 📍 Seacliff CD, Hyatt Regency San Francisco
5 Embarcadero Center, San Francisco, CA 94111

[Register for Workshop](#)

So now what?

Many of us enjoy the craft of
building software.

Technology changes.

Constantly.

Our industry is evolving.

Some will be tempted to just...



It's a bold strategy Cotton.
Let's see if it pays off.

Two possible mindsets.

You can define yourself by what
you've done in the past.

Or...

You can define yourself by the
problems you will solve in the future.

You can be reactive.

Or proactive.

Whatever path you choose,
change is inevitable.

You are responsible
for your career!

No one can do it for you.

Fundamentals will
always serve you well.

“Excellence is doing ordinary things extraordinarily well.”

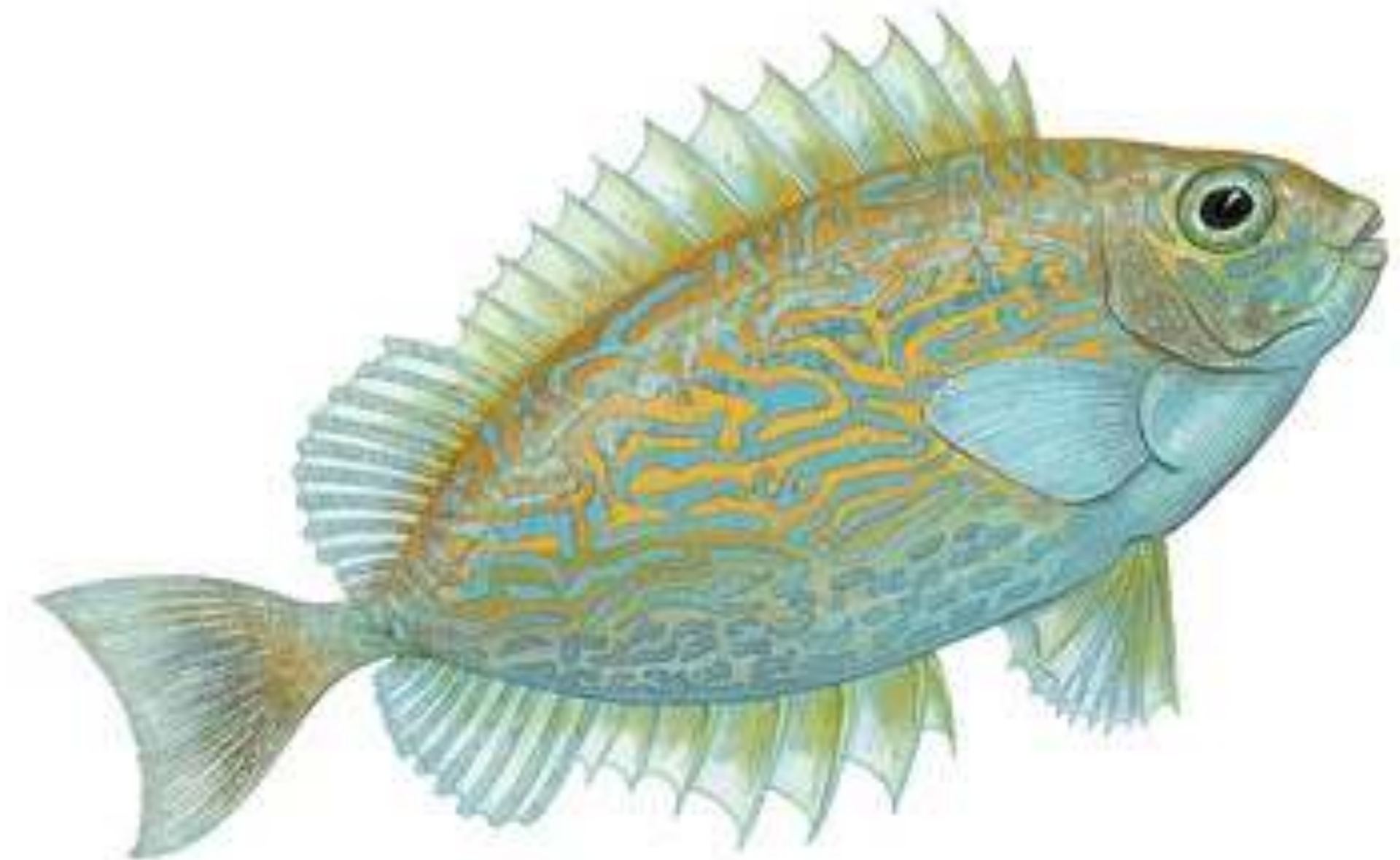
– John W. Gardner

Good luck!

O'REILLY

Fundamentals of Software Engineering

From Coder to Engineer



Nathaniel Schutta
& Dan Vega

<https://fundamentalsofswe.com>

Code: FOSEQCON25

Link: [https://learning.oreilly.com/
get-learning/?code=FOSEQCON25](https://learning.oreilly.com/get-learning/?code=FOSEQCON25)



Nathaniel Schutta

🦋 @nts.bsky.social

<https://www.linkedin.com/in/nate-schutta>

<https://ntschutta.io/>



Dan Vega

X @therealdanvega

<https://www.linkedin.com/in/danvega>

<https://danvega.dev>

